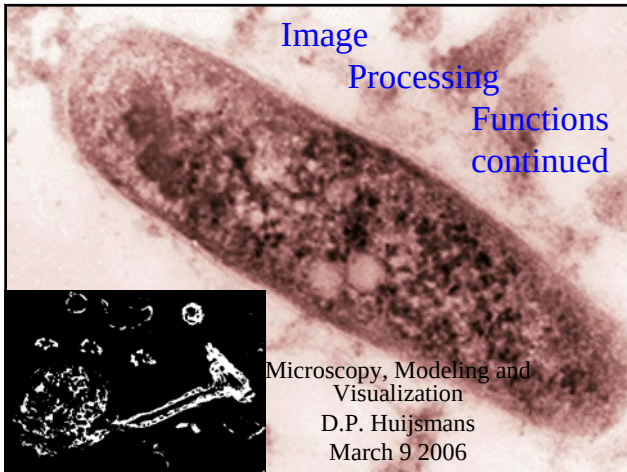




Computer Imagery			
Input \ Output	Analog Image	Digital Image	Non-Image datastructure
Analog Image	Optics	Discretization /Sampling	Image Interpretation
Digital Image	Image Reconstruction	Image Processing	Image Analysis
Non-Image datastructure	Drawing Painting	Computer Graphics	Rest of Computer Science

Neighborhood Filter Classes		
Location center pixel \ Pixel values involved	Input center pixel location	Coordinate transformation
Only intensity values	Rank Filter	
Both intensity and location values	Convolution Filter	Geometrical Transformation with Intensity Interpolation
	Morphological Transform	

Rank (or Percentile) Filter statistical filter

Intensities in corresponding kxl neighborhood are **sorted**:

Certain percentile value transferred to output:

0% min filter, 50% **median filter**, 100% max filter

Min an max filter emphasize outliers;

Median filter is a very good noise reduction algorithm

Repeated application of the median filter produces an image with a painted appearance

Rank Filter example

Around $\text{Int}_{in}(i,j)=30$ 0 50 100 %

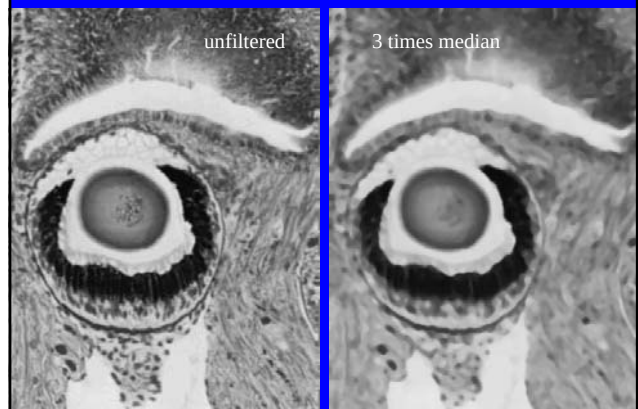
20	27	40
40	30	37
50	42	35

20	27	30	35	37	40	40	42	50
----	----	----	----	----	----	----	----	----

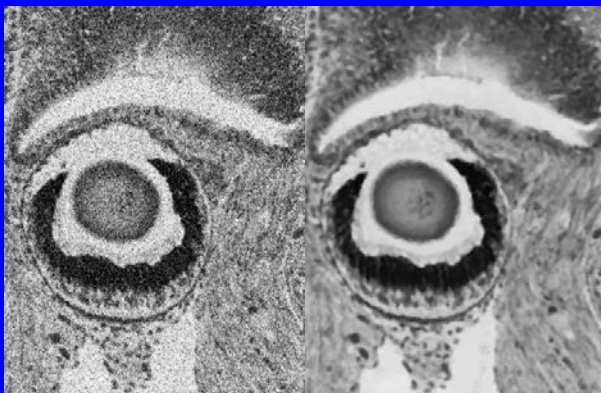
Min Median Max

Intout (i,j) = 20 Min
37 Med
50 Max

Rank Filter example



Noise removal with Median filter



more complex statistical filters

- **MAX-MIN filter (Range filter):** largest local difference; sort of gradient image
- **Kuwahara Filter:** This filter is an edge-preserving filter. In each of four compass regions (NW,NE,SE,SW), the mean brightness and the variance are calculated. The output value of the center pixel in the window is the mean value of that compass region that showed the smallest variance.
This filter is an edge-preserving filter, which smoothes the images without disturbing the sharpness and the position of edges.

Kuwahara Filter

nw	nw	n	ne	ne
nw	nw	n	ne	ne
w	w	c	e	e
sw	sw	s	se	se
sw	sw	s	se	se



Convolution or Weight Filter

Output value is a **weighted sum of intensity values** around the corresponding input pixel location (convolution)

The weights can depend on the location in the neighborhood

I1	I2	I3
I4	I5	I6
I7	I8	I9

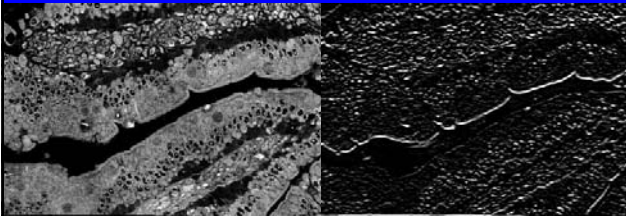
W1	W2	W3
W4	W5	W6
W7	W8	W9

3x3 Weight Coefficient Mask

$$O5 = I1*W1 + I2*W2 + + I9*W9 \text{ (Dot Product)}$$

Gradient Filter or Edge Detector

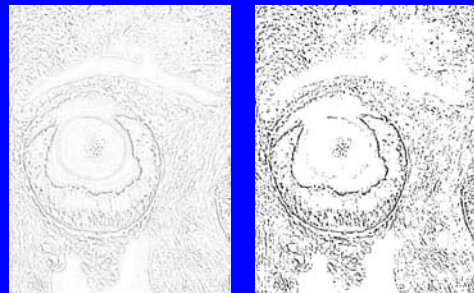
An important class of convolution filters are first- or second derivative filters that produce low values in uniform areas and high values in local areas with large intensity changes.



Horizontal edge magnitude

Thresholding Edge Maps

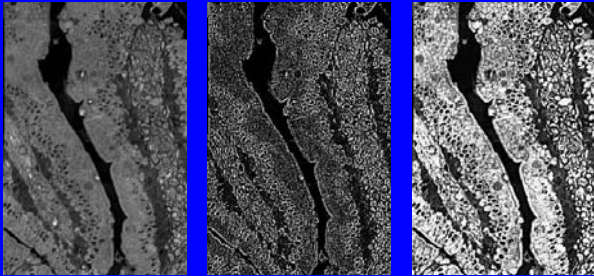
A drawing like binary image is obtained by a threshold in the inverted gradient image



Gradients often coincide with object edges (but also with shadow edges!)

Image Sharpening

Adding a scaled gradient image to the original enhances local contrasts



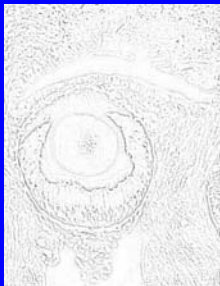
Some sharpening should be applied when the original image is smoothed by down sampling resolution or excessive compression

Unsharp
masking with V-
channel for color
image



Positive and negative gradients

As in mountainous areas hills can go steep up or steep downwards



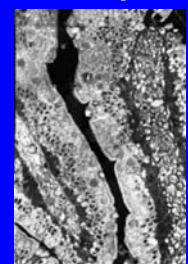
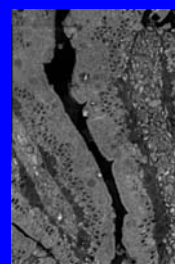
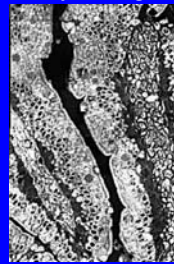
Absolute value of inverted
gradient image



Signed gradient values
around gray level halfway

Smoothing Filters

By blurring local contrast can be softened instead of sharpened



-1	0	1
-2	1	2
-1	0	1

0	0	0
0	1	0
0	0	0

1	2	1
2	4	2
1	2	1

*1/16

Image Processing

Transforms 1 or more input images into an output image

Intensity domain Transformations:

- point operations
- algebraic operations
- logic operations
- rank filter
- convolution filter
- geometrical transformation
- morphological transformation

Spatial Frequency domain Transformations (later)

Image Transformations

Parameters:

- Number of input matrices (mostly 1 or 2)
- Pixel Intensity values
- Pixel coordinate values (i,j or x,y)

$$Int_{out}(i,j) = F(Int_{in1}(k,l), Int_{in2}(k,l), \dots; Loc1(k,l), Loc2(k,l), \dots)$$

i,j,k,l in range [1,n]

Global recipe (algorithm) for calculating output pixel intensity and location from input intensities and/or locations

Neighborhood Filter Classes

Location center pixel Pixel values involved	Input center pixel location	Coordinate transformation
Only intensity values	Rank Filter	
Both intensity and location values	Convolution Filter Morphological Transform	Geometrical Transformation with Intensity Interpolation

Geometrical Transformation

Coordinate transformation: intensity value moves to new position

Affine transform:
Translation (shift)
Rotation, scaling,
Perspective proj.



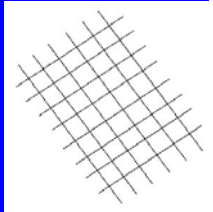
Rubber sheet transform:
Non-linear transforms



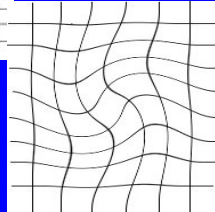
Geometrical Transformation grid

Coordinate transformation: intensity value moves to new position

Global transform:
Affine transform
Straight lines remain
straight



Rubber sheet transform:
Straight lines become
curved



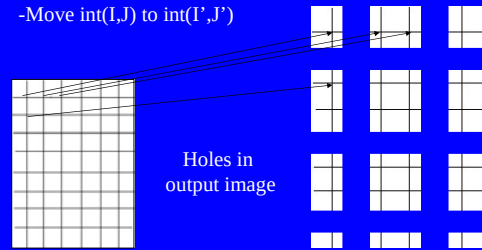
Geometrical Transform forward

Forward Transform (Pixel Carryover):

-Recipe for calculating new position X, Y given old position I, J
(for example $X=1.5*I, Y=1.5*J$)

-Truncate REAL X, Y to INTEGER I', J'

-Move $\text{int}(I, J)$ to $\text{int}(I', J')$



Geometrical Transform inverse

Pixel fill-in:

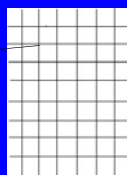
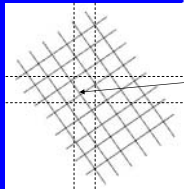
For each output position I', J' :

-Calculate REAL X, Y position in input image
using inverse coordinate transform

-determine output intensity with

interpolation scheme

from nearby input intensities
at integer positions



Interpolation Scheme

$I, J+1$

$I+1, J+1$

X, Y

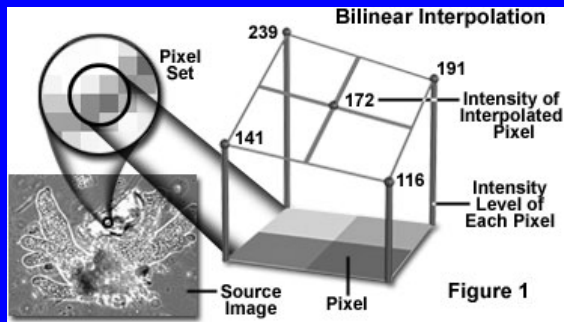
I, J

$I+1, J$

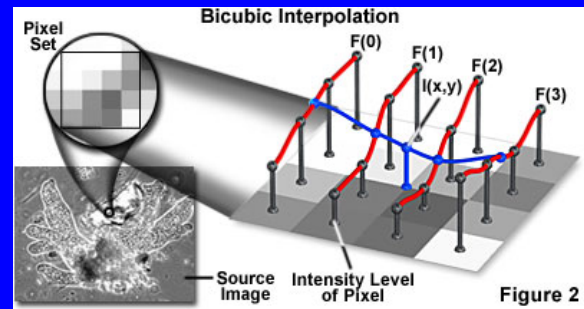
$I = \text{INT}(X), J = \text{INT}(Y)$

Output position comes from Input position X, Y
which is between 4 grid intensities
What intensity should be filled in?

Bilinear interpolation



Bicubic interpolation



Bilinear Interpolation

Output intensity determined by distance-weighted sum of 4 (2x2) surrounding input intensities

Bicubic Interpolation

Output intensity determined by form of bicubic surface
Determined by 16 (4x4) surrounding input intensities

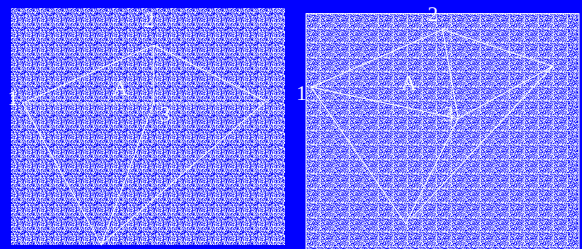
Nearest Neighbour value

Output intensity is determined by rounding X,Y to integer

Corner value

Output intensity is determined by truncating X,Y to Integer

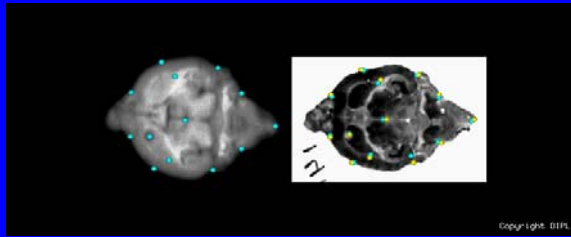
Image Warping with corresponding points: piecewise affine transformations



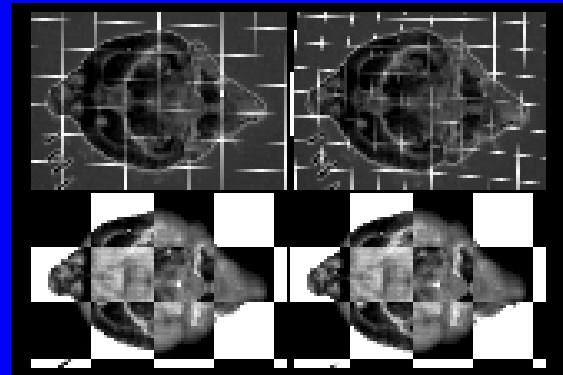
Use corresponding points to triangulate the images:
now for each pair of corresponding triangles a coordinate transformation can be found on the basis of which intensities move to a new position

Episcopic and diascopic series

- Correction for deformation due to slicing, swelling
- and mounting by recording uncut top views



Checkerboard overlay



Discrete Geometry

- Sampled values
- Point, time samples versus image sampling
- Regular grids
- Local connectivity
- Labelling algorithm
- Contour extraction algorithm

Image Sampling

Image sample:

- discrete intensity value
- 2D location values

Light collected within a (small) window within
a (small) time interval

Not a point sample (not in time, not in space)!!

Point sampling versus Image sampling

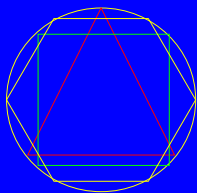
- Time series: 1D
- Point sampling, T_i
- Pos. and neg. values
- Superposition principle
- Cause-effect relation
- Frequency domain view
- Image matrix: 2D
- Integration area, dT
- Only pos. values
- No-superposition
- Cohesion
- Space domain view
- Less aliasing
- Better signal to noise

Linear Algebra
Theory

Sampling windows

- Centers make up a grid
- Windows may (or may not) overlap
- How well is image area covered?
- How efficiently is image area covered?
- What grid structure is to be preferred ?
- Consequences for local topology

Ideal Sampling Window and 2D space fillers



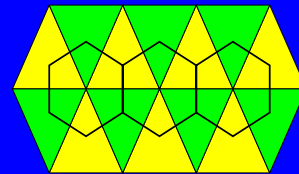
Circular window ideal (isotropic in 2D), but not a space filler:

2D space fillers: equilateral triangle, square, hexagon

Each space filler defines a different local connectivity

Local connectivity = number of nearest neighbours

Sampling window and grid cell for regular triangle

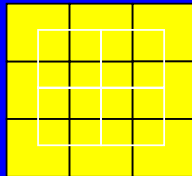


Connectivity = 3

Grid cell is hexagon

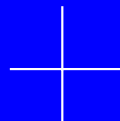
Implicit pixel position: after 2 lines alternating $\pm \frac{1}{2}$ grid size

Square Sampling window and grid cell



Grid cell also square

Implicit pixel position: same for each line: grid size * (i,j)



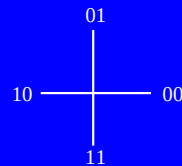
Connectivity = 4



Connectivity = 8?

Square grid: contour codes

Connectivity = 4



Crack-code:

4 directions

2 bit/contour point

Connectivity = 8

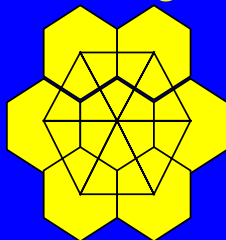


Chain-code:

8 directions

3 bit/contour point

Sampling window and grid cell for hexagon



Grid cell is regular triangle

Hexagon is most efficient sampling cell

Implicit pixel position: next line alternating $\pm \frac{1}{2}$ grid size



Connectivity = 6

Foreground, background, contours and connectivity

```
000000000000
011110000000
001111111000
001111111100
000011111000
000000000000
```

Path of incremental nearest neighbor connections

```
000000000000
011110000000
001011111000
001110001100
000011111000
000000000000
```

4-connected closed contour:
two 4-connected inner areas

```
000000000000
011110000000
001001111100
00?100000100
000011111000
000000000000
```

8-connected closed contour:
but 8-connected inner area
8-connected to bg!

Square grids: hybrid connectivity scheme

- 8-connected contours and areas conflict:
- 8-connected contours & 4-connected areas
- 4-connected contours and 8-connected areas
- 4-connected contours and areas
- Consequences for algorithms at pixel level!!

4-connected area label algorithm

Binary image: 0 = bg ; 1 = fg

L-template with C=current position, X , Y visited pixels



$C = \{0,1\}$, $X = \{0, m\}$, $Y = \{0, k\}$

At start area count = 0; equal areas table empty;

Phase 1: For each pixel from upper-left to lower-right

-case $C = 0$: continue

- case $Y = X = 0$: $m = m+1$ $C = m$ continue

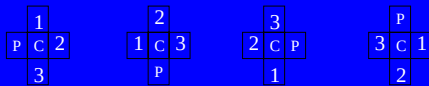
- case $Y = X = k$: $C = k$ continue

- case $Y=k$, $X = m$: $C = \min(k,m)$; merge labels $k=m$

Phase 2: revisit pixels :keep lowest number of merged labels

Question? What is a handy initialization for this template?

4-connected contour follow algorithm



Given binary image with one 4-connected area not touching edge

Search phase: Visit pixels from ul to lr until fg pixel found:

-current fg-pixel coordinates start-point closed contour = C;

- previous pixel = P;

Trace phase: until first 2 contour-coordinates repeat:

-search for fg-pixel clockwise w.r.t. PC direction;

-Add next fg-pixel position to contour;

- current fg-pixel = C, previous = P